IMPLEMENTASI STRUKTUR DATA QUEUE (ANTRIAN)

Mata Kuliah :

Struktur Data

Oleh :

Muhammad Pasha Herianto

250913970363

2025B

PROGRAM STUDI MANAJEMEN INFORMATIKA

FAKULTAS VOKASI

UNIVERSITAS NEGERI SURABAYA

2026

1. Tentukan kompleksitas waktu (worst case) untuk setiap operasi yang didefinisikan dalam class TicketCounterSimulation.

Jawaban:

Dalam struktur data queue, terdapat beberapa operasi utama yang digunakan untuk mengelola data dalam antrian. Setiap operasi memiliki tingkat efisiensi yang diukur dengan kompleksitas waktu.

Operasi pada Queue:

a. Enqueue (menambah data)
   - Fungsi: menambahkan elemen ke bagian belakang antrian
   - Cara kerja: langsung menambah ke posisi akhir
   - Kompleksitas: O(1)
   - Penjelasan: tidak perlu menggeser data lain, jadi prosesnya cepat dan tetap

b. Dequeue (menghapus data)
   - Fungsi: menghapus elemen dari bagian depan antrian
   - Cara kerja: mengambil elemen pertama
   - Kompleksitas: O(1)
   - Penjelasan: hanya mengambil satu elemen tanpa mencari atau menggeser data

c. isEmpty (cek kosong atau tidak)
   - Fungsi: mengecek apakah antrian kosong
   - Kompleksitas: O(1)
   - Penjelasan: hanya mengecek jumlah data, tidak perlu perulangan

d. Melihat isi queue (print / akses)
   - Kompleksitas: O(n)
   - Penjelasan: karena harus menampilkan semua elemen satu per satu

2. Lakukan eksekusi manual kode berikut dan tampilkan isi queue yang dihasilkan:

```
print("=== Jawaban Nomor 2 ===")
values = Queue()
for i in range(16):
    if i % 3 == 0:
        values.enqueue(i)
print("Isi Queue:", values)
```

```
=== Jawaban Nomor 2 ===
Isi Queue: [0, 3, 6, 9, 12, 15]
```

Analisis Hasil:

Program diatas melakukan perulangan dari 0 sampai 15, kemudian hanya memasukkan angka yang habis dibagi 3 ke dalam queue. Data yang memenuhi kondisi akan masuk ke dalam antrian tanpa ada penghapusan, sehingga urutannya tetap sesuai dengan urutan masuk.

3. Lakukan eksekusi manual kode berikut dan tampilkan isi queue yang dihasilkan:

```
print("\n=== Jawaban Nomor 3 ===")
values = Queue()
for i in range(16):
    if i % 3 == 0:
        values.enqueue(i)
    elif i % 4 == 0:
        values.dequeue()
print("Isi Queue:", values)
```

```
=== Jawaban Nomor 3 ===
Isi Queue: [6, 9, 12, 15]
```

Analisis Hasil:

Program diatas memiliki dua kondisi, yaitu menambah data jika habis dibagi 3 dan menghapus data jika habis dibagi 4. Proses ini menyebabkan beberapa elemen awal keluar dari antrian, sehingga hasil akhirnya hanya menyisakan beberapa data.

4. Implementasikan metode yang tersisa dari class TicketCounterSimulation.

```python
print("\n=== Jawaban Nomor 4 ===")
q = Queue()
q.enqueue(10)
q.enqueue(20)
q.enqueue(30)
print("Sebelum dequeue:", q)
q.dequeue()
print("Sesudah dequeue:", q)
```

```
=== Jawaban Nomor 4 ===
Sebelum dequeue: [10, 20, 30]
Sesudah dequeue: [20, 30]
```

Analisis Hasil:

Dari program diatas bisa dilihat bahwa data pertama yang masuk akan keluar lebih dulu saat dilakukan dequeue. Hal ini menunjukkan bahwa queue bekerja dengan konsep FIFO (First In First Out).

5. Modifikasi kelas TicketCounterSimulation agar menggunakan satuan detik, bukan menit. Jalankan eksperimen dan buat tabel seperti Tabel 8.1.

Kasus 1: 2 Agents, Average Service = 3 detik						
Num Seconds	**Num Agents**	**Avg Service**	**Time Between**	**Avg Wait**	**Served**	**Remaining**
100	2	3	2	2.49	49	2
500	2	3	2	3.91	240	0
1000	2	3	2	10.93	490	14
5000	2	3	2	15.75	2459	6
10000	2	3	2	21.17	4930	18

Kasus 2: 2 Agents, Average Service = 4 detik						
Num Seconds	**Num Agents**	**Avg Service**	**Time Between**	**Avg Wait**	**Served**	**Remaining**
100	2	4	2	10.60	40	11
500	2	4	2	49.99	200	40
1000	2	4	2	95.72	400	104
5000	2	4	2	475.91	2000	465
10000	2	4	2	949.61	4000	948

Kasus 3: 3 Agents, Average Service = 4 detik						
Num Seconds	**Num Agents**	**Avg Service**	**Time Between**	**Avg Wait**	**Served**	**Remaining**
100	3	4	2	0.51	51	0
500	3	4	2	0.50	240	0
1000	3	4	2	01.06	501	3
5000	3	4	2	1.14	2465	0
10000	3	4	2	1.21	4948	0

Analisis Hasil:

Setelah dimodifikasi ke satuan detik, hasil simulasi tetap menunjukkan pola yang sama seperti versi menit, yaitu:

- Waktu pelayanan sangat berpengaruh terhadap antrian. Semakin lama waktu pelayanan (misalnya dari 3 detik menjadi 4 detik), maka antrian akan semakin panjang dan waktu tunggu pelanggan meningkat.
- Jumlah petugas (agents) juga mempengaruhi kinerja sistem. Dengan 2 petugas, antrian masih bisa menumpuk, sedangkan dengan 3 petugas, antrian menjadi jauh lebih cepat berkurang.
- Waktu tunggu (average wait) meningkat drastis ketika pelayanan lebih lama, tetapi bisa ditekan menjadi sangat kecil jika jumlah petugas ditambah.
- Sistem antrian dapat mengalami overload jika waktu pelayanan terlalu lama dan tidak diimbangi dengan jumlah petugas yang cukup.
- Semakin lama waktu simulasi (num seconds), semakin banyak pelanggan yang diproses. Namun jika pelayanan lambat, jumlah pelanggan yang belum terlayani juga akan terus bertambah.
- Secara keseluruhan, keseimbangan antara waktu pelayanan dan jumlah petugas sangat penting agar sistem antrian berjalan optimal dan tidak terjadi penumpukan.

6. Rancang dan implementasikan fungsi untuk membalik urutan item dalam queue. Hanya boleh menggunakan operasi Queue ADT (boleh pakai struktur data lain).

```python
print("\n=== Jawaban Nomor 6 ===")
def reverseQueue(q):
    stack = []
    while not q.isEmpty():
        stack.append(q.dequeue())
    while stack:
        q.enqueue(stack.pop())
    return q

q = Queue()
for i in [1, 2, 3, 4]:
    q.enqueue(i)

print("Sebelum reverse:", q)
reverseQueue(q)
print("Sesudah reverse:", q)
```

```
=== Jawaban Nomor 6 ===
Sebelum reverse: [1, 2, 3, 4]
Sesudah reverse: [4, 3, 2, 1]
```

Analisis Hasil:

Queue dapat dibalik dengan bantuan stack. Hal ini karena stack menggunakan konsep LIFO (Last In First Out), yaitu data yang terakhir masuk akan keluar lebih dulu. Prosesnya dilakukan dengan memindahkan semua elemen dari queue ke stack sehingga urutannya terbalik, lalu memindahkannya kembali ke queue. Karena stack membalik urutan saat proses tersebut, maka hasil akhirnya queue akan memiliki urutan yang terbalik dari sebelumnya.